Contents

1 Data Structures

1.1 Cht

```
typedef int ftype;
typedef complex<ftype> point;
#define x real
#define y imag
ftype dot(point a, point b) {
return (conj(a) * b).x();
}
ftype cross(point a, point b) {
return (conj(a) * b).y();
}
vector<point> hull, vecs;
void add_line(ftype k, ftype b) {
point nw = {k, b};
while(!vecs.empty() && dot(vecs.back(), nw - hull.back()) < 0) {
hull.pop_back();
vecs.pop_back();
}
if(!hull.empty()) {
vecs.push_back(1i * (nw - hull.back()));
}
hull.push_back(nw);
}
int get(ftype x) {
point query = {x, 1};
auto it = lower_bound(vecs.begin(), vecs.end(), query, [](point a, point
    b) {
return cross(a, b) > 0;
});
return dot(query, hull[it - vecs.begin()]);
}
```

1.2 Dynamic Segment Tree

```
struct Vertex {
int left, right;
int sum = 0;
Vertex *left_child = nullptr, *right_child = nullptr;
Vertex(int lb, int rb) {
left = lb;
```

```
right = rb;
}
void extend() {
if (!left_child && left + 1 < right) {
int t = (left + right) / 2;
left_child = new Vertex(left, t);
right_child = new Vertex(t, right);
}
}
void add(int k, int x) {
extend();
sum += x;
if (left_child) {
if (k < left_child->right)
left_child->add(k, x);
else
right_child->add(k, x);
}
}
int get_sum(int lq, int rq) {
if (lq <= left && right <= rq)
return sum;
if (max(left, lq) >= min(right, rq))
return 0;
extend();
return left_child->get_sum(lq, rq) + right_child->get_sum(lq, rq);
}
};
```

1.3 LiChao Tree

```
typedef ll ftype;
typedef complex<ftype> point;
#define x real
#define y imag
const ll MOD=1e9+7; //998244353LL;
const ll INF=1LL<<60;
const int maxn = 2e5;
point line[4 * maxn];
ftype dot(point a, point b) {
return (conj(a) * b).x();
}
ftype f(point a, ftype x) {
return dot(a, {x, 1});
}
void add_line(point nw, int v, int l, int r, int
lf, int rf) {
int m = (l + r) / 2;
if(rf < l || r <= lf) return;
if(l < lf || rf < r-1){
add_line(nw, 2 * v, l, m, lf, rf);
add_line(nw, 2 * v + 1, m, r, lf, rf);
return;
}
bool lef = f(nw, l) < f(line[v], l);
bool mid = f(nw, m) < f(line[v], m);
if(!mid) {
swap(line[v], nw);
}
if(r - l == 1) {
return;
} else if(lef != mid) {
add_line(nw, 2 * v, l, m, lf, rf);
} else {
add_line(nw, 2 * v + 1, m, r, lf, rf);
}
}
ftype get(int x, int v, int l, int r) {
int m = (l + r) / 2;
if(r - l == 1) {
return f(line[v], x);
} else if(x < m) {
return max(f(line[v], x), get(x, 2 * v,
l, m));
} else {
return max(f(line[v], x), get(x, 2 * v +
1, m, r));
}
```

1.4 Persistent

```
struct Vertex {
Vertex *l, *r;
int sum;
Vertex(int val) : l(nullptr), r(nullptr), sum(val) {}
Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum(0) {
```

```
if (l) sum += l->sum;
if (r) sum += r->sum;
}
};
Vertex* build(int a[], int tl, int tr) {
if (tl == tr)
return new Vertex(a[tl]);
int tm = (tl + tr) / 2;
return new Vertex(build(a, tl, tm), build(a, tm+1, tr));
}
int get_sum(Vertex* v, int tl, int tr, int l, int r) {
if (l > r)
return 0;
if (l == tl && tr == r)
return v->sum;
int tm = (tl + tr) / 2;
return get_sum(v->l, tl, tm, l, min(r, tm))
+ get_sum(v->r, tm+1, tr, max(l, tm+1), r);
}
Vertex* update(Vertex* v, int tl, int tr, int pos, int new_val) {
if (tl == tr)
return new Vertex(new_val);
int tm = (tl + tr) / 2;
if (pos <= tm)
return new Vertex(update(v->l, tl, tm, pos, new_val), v->r);
else
return new Vertex(v->l, update(v->r, tm+1, tr, pos, new_val));
}
```

2 Graphs

2.1 Mcmf

```
using T = long long;
const T inf = 1LL << 61;
struct MCMF {
  struct edge {
    int u, v;
    T cap, cost;
    int id;
    edge(int _u, int _v, T _cap, T _cost, int _id) {
      u = _u;
      v = _v;
      cap = _cap;
      cost = _cost;
      id = _id;
    }
  };
  int n, s, t, mxid;
  T flow, cost;
  vector<vector<int>> g;
  vector<edge> e;
  vector<T> d, potential, flow_through;
  vector<int> par;
  bool neg;
  MCMF() {}
  MCMF(int _n) { // 0-based indexing
    n = _n + 10;
    g.assign(n, vector<int> ());
    neg = false;
    mxid = 0;
  }
  void add_edge(int u, int v, T cap, T cost, int id = -1, bool directed
      = true) {
    if(cost < 0) neg = true;
    g[u].push_back(e.size());
    e.push_back(edge(u, v, cap, cost, id));
    g[v].push_back(e.size());
    e.push_back(edge(v, u, 0, -cost, -1));
    mxid = max(mxid, id);
    if(!directed) add_edge(v, u, cap, cost, -1, true);
  }
  bool dijkstra() {
    par.assign(n, -1);
    d.assign(n, inf);
    priority_queue<pair<T, T>, vector<pair<T, T>>, greater<pair<T, T>> >
        q;
    d[s] = 0;
    q.push(pair<T, T>(0, s));
    while (!q.empty()) {
      int u = q.top().second;
      T nw = q.top().first;
      q.pop();
      if(nw != d[u]) continue;
      for (int i = 0; i < (int)g[u].size(); i++) {
        int id = g[u][i];
```

```
        int v = e[id].v;
        T cap = e[id].cap;
        T w = e[id].cost + potential[u] - potential[v];
        if (d[u] + w < d[v] && cap > 0) {
          d[v] = d[u] + w;
          par[v] = id;
          q.push(pair<T, T>(d[v], v));
        }
      }
    }
    for (int i = 0; i < n; i++) {
      if (d[i] < inf) d[i] += (potential[i] - potential[s]);
    }
    for (int i = 0; i < n; i++) {
      if (d[i] < inf) potential[i] = d[i];
    }
    return d[t] != inf; // for max flow min cost
    // return d[t] <= 0; // for min cost flow
  }
  T send_flow(int v, T cur) {
    if(par[v] == -1) return cur;
    int id = par[v];
    int u = e[id].u;
    T w = e[id].cost;
    T f = send_flow(u, min(cur, e[id].cap));
    cost += f * w;
    e[id].cap -= f;
    e[id ^ 1].cap += f;
    return f;
  }
  //returns {maxflow, mincost}
  pair<T, T> solve(int _s, int _t, T goal = inf) {
    s = _s;
    t = _t;
    flow = 0, cost = 0;
    potential.assign(n, 0);
    if (neg) {
      // Run Bellman-Ford to find starting potential on the starting
          graph
      // If the starting graph (before pushing flow in the residual
          graph) is a DAG,
      // then this can be calculated in O(V + E) using DP:
      // potential(v) = min({potential[u] + cost[u][v]}) for each u -> v
          and potential[s] = 0
      d.assign(n, inf);
      d[s] = 0;
      bool relax = true;
      for (int i = 0; i < n && relax; i++) {
        relax = false;
        for (int u = 0; u < n; u++) {
          for (int k = 0; k < (int)g[u].size(); k++) {
            int id = g[u][k];
            int v = e[id].v;
            T cap = e[id].cap, w = e[id].cost;
            if (d[v] > d[u] + w && cap > 0) {
              d[v] = d[u] + w;
              relax = true;
            }
          }
        }
      }
      for(int i = 0; i < n; i++) if(d[i] < inf) potential[i] = d[i];
    }
    while (flow < goal && dijkstra()) flow += send_flow(t, goal - flow);
    flow_through.assign(mxid + 10, 0);
    for (int u = 0; u < n; u++) {
      for (auto v : g[u]) {
        if (e[v].id >= 0) flow_through[e[v].id] = e[v ^ 1].cap;
      }
    }
    return make_pair(flow, cost);
  }
};
```

3 Math

3.1 Or conv

```
void solve(){
int n,k; cin>>n>>k;
vector<ll> acu(1<<k), sos(1<<k);
string s;
for(int i=0; i<n ; i++){
cin>>s;
sos[stoi(s,nullptr, 2)]++;
}
```

```
for (int i = 0; i < k; i++) {
for (int j = 0; j < (1 << k); j++) {
if ((j >> i) & 1) {
sos[j] += sos[j - (1 << i)];
}
}
}
for(int i=0; i<(1<<k); i++) {
if(i<3) acu[i]=0;
acu[i] = ((sos[i])*(sos[i]-1)*(sos[i]-2))/6;
}
for (int i = k - 1; i >= 0; i--) {
for (int j = (1 << k) - 1; j >= 0; j--) {
if ((j >> i) & 1) {
acu[j] -= acu[j - (1 << i)];
}
}
}
int m; cin>>m;
for(int i=0; i<m; i++){
cin>>s;
cout<<acu[stoi(s,nullptr, 2)]<<endl;
}
}
```